

Q1 Final Examination, 7th August 2026. 50 marks, 2 hours. Teacher's copy — worked solutions and marking notes.

Part A — Multiple Choice Questions

8 marks

Q1

[1]

Answer. (b) Dynamic Host Configuration Protocol

Working. DHCP is the protocol that hands out IP addresses automatically when a device joins a network, instead of someone typing one in by hand. The other three options are invented names.

Q2

[1]

Answer. (b) Ctrl + C, then Ctrl + V

Working. Ctrl + C copies (the original stays), Ctrl + V pastes. Option (a) starts with Ctrl + X, which *cuts* – the original is removed, so that is move, not copy. Ctrl + P is print and Ctrl + S is save.

Q3

[1]

Answer. (c) 172.16.300.5

Working. An IPv4 address is four numbers, each stored in one byte, so each one must be in the range **0 to 255**. In (c) the third number is **300**, which is bigger than 255, so it cannot be stored.

Note (b) 10.0.0.255 *is* valid – 255 is a legal value for a byte. Rejecting it is the common mistake.

Q4

[1]

Answer. (b) $\overline{A} \cdot \overline{B}$

Working. Read the circuit: A and B go into an **OR** gate, and the result goes into a **NOT** gate. So $Y = \overline{A + B}$ (a NOR gate).

Now apply **De Morgan's law** – the bar over a sum becomes a product of bars:

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Checking with a truth table, Y is 1 only when both inputs are 0:

A	B	$\overline{A + B}$	$\overline{A} \cdot \overline{B}$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

The two columns match. Option (a) $\overline{A} + \overline{B}$ is the **NAND** (1,1,1,0) – the classic De Morgan slip of moving the bar in without changing the operator.

Q5

[1]

Answer. (c) 128

Working. Each bit has 2 possible values, and there are 7 independent bits, so the number of different patterns is

$$2^7 = 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = \mathbf{128}$$

(a) 7 is the bit *count*, not the number of patterns; (b) 64 is 2^6 ; (d) 256 is 2^8 – that is a whole byte, but ASCII uses only 7 of those 8 bits.

Q6

[1]

Answer. (b) The text `A1+A2`

Working. A spreadsheet only treats what you type as a **formula** if it begins with an **equals sign**. `A1+A2` has no `=`, so Calc has nothing to calculate and stores it as ordinary text. Typing `=A1+A2` would give the sum.

Q7

[1]

Answer. (c) `.zip`

Working. A `.zip` file is an *archive*: it holds many files inside one file, so all 50 travel as a single attachment (and compressed, so it is smaller too). `.png` and `.jpg` are single images and `.txt` is a single text file – none of them can contain other files.

Q8

[1]

Answer. (c) 0

Working. $\bar{A}$ is always the opposite of A , so the two can never be 1 at the same time, and AND needs *both* inputs to be 1:

$$\text{if } A = 0: 0 \cdot \bar{0} = 0 \cdot 1 = 0$$

$$\text{if } A = 1: 1 \cdot \bar{1} = 1 \cdot 0 = 0$$

Either way the answer is **0**. This is the **complement law**, $A \cdot \bar{A} = 0$.

Part B — Short Answer Questions

8 marks

Q9

[1]

Answer. See the circuit below.

Working. Work outwards from the brackets in $Y = (a \cdot b) + \bar{c}$:

1. An **AND** gate with inputs a and b (gives $a \cdot b$).

2. A **NOT** gate on c (gives $\bar{c}$).

3. An **OR** gate taking those two outputs, giving Y .

Three gates in total. **1 mark** for a correct circuit; the NOT must sit on c alone, not on the whole expression.

Q10

[1]

Answer. `B2:C3`

Working. Read the shaded block off the grid: it starts at the top-left shaded cell and ends at the bottom-right one. Top-left is column **B**, row **2**; bottom-right is column **C**, row **3**. A range is written `first:last`, so `B2:C3` (4 cells). Accept lower case.

Q11

[1]

Answer. `Ctrl + Z`

Working. `Ctrl + Z` undoes the last action. (`Ctrl + Y` redoes it.) Accept `Cmd + Z` on a Mac.

Q12

[1]

Answer. 24 bits

Working. The trap is that the question gives *bytes* and asks for *bits*, so two steps are needed:

1. RGB means three colours – Red, Green, Blue – so one pixel uses **3 bytes**.
2. 1 byte = 8 bits, so $3 \times 8 = \mathbf{24 \text{ bits}}$.

Answering “3” means missing the unit change.

Q13

[1]

Answer. 2

Working. Two different operators, worked out separately:

- $23 // 4$ is *whole-number division* – how many whole 4s fit into 23. $4 \times 5 = 20$ and $4 \times 6 = 24$, so it is **5** (the remainder is thrown away).
- $23 \% 4$ is the *remainder* after that division: $23 - 20 = \mathbf{3}$.

Then $5 - 3 = \mathbf{2}$.

Note the subtraction: swapping the two operators gives $3 - 5 = -2$, so a student who mixes them up cannot land on the right answer by accident.

Q14

[1]

Answer. (b) Application software

Working. **Application** software is written for the *user* to do something with – play, write, draw, calculate. **System** software runs the machine itself. A game is something you use the computer *for*, so it is an application.

(d) is the trap: an operating system is a kind of *system* software, so anyone half-remembering the pair “application / system” can be pulled towards it.

Q15

[2]

Answer. B8 $\rightarrow$ **1600** B9 $\rightarrow$ **1400**

Working. =SUM(B2:B7) adds every price in the range:

$$40 + 150 + 850 + 250 + 10 + 300 = \mathbf{1600}$$

=SUMIF(B2:B7, ">200") adds only the prices that are *greater than* 200. Check each one: 40 no, 150 no, 850 **yes**, 250 **yes**, 10 no, 300 **yes**.

$$850 + 250 + 300 = \mathbf{1400}$$

1 mark each. The skill being tested is deciding *which* numbers to add, not the adding.

Part C — Boolean Algebra

6 marks

Q16

[3]

Answer. Truth table below. For the two cases asked: $A=1, B=0, C=1 \Rightarrow Y = \mathbf{1}$; $A=1, B=1, C=0 \Rightarrow Y = \mathbf{0}$.

Working. $Y = (A \cdot \overline{B} \cdot C) + (\overline{A} \cdot B)$ is an OR of two terms, so $Y = 1$ when *either* term is 1.

- First term $A \cdot \overline{B} \cdot C$ needs $A=1, B=0$ and $C=1$ all at once – true on one row only.
- Second term $\overline{A} \cdot B$ needs $A=0$ and $B=1$, whatever C is – true on two rows.

So exactly 3 of the 8 rows give 1. Build the table with A, B, C counting up in binary 000 to 111 so no row is missed:

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

Checking the two asked cases against it: row $A=1, B=0, C=1$ is the first term satisfied, so $Y = 1$; row $A=1, B=1, C=0$ fails both terms (B is 1 so the first dies, A is 1 so the second dies), so $Y = 0$.

3 marks: 1 for the 8 input rows in order, 2 for the output column (or 1 if only one or two rows are wrong).

Q17

[3]

Answer. See the circuit below.

Working. $Y = (a + b) \cdot \bar{c}$. Brackets first, so the OR happens before the AND:

1. An **OR** gate with inputs a and $b \rightarrow a + b$.
2. A **NOT** gate on $c \rightarrow \bar{c}$.
3. An **AND** gate taking those two $\rightarrow Y$.

3 marks: 1 per gate, correctly connected. The commonest error is feeding c straight into the AND and putting the NOT at the end, which draws $\overline{(a + b) \cdot c}$ instead.

a	b	c	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

(Truth table given for checking – it was not asked for.)

Part D — Number Systems

5 marks

Q18

[2]

Answer. $1011010_2 = 90_{10}$

Working. Write the place values above the bits, doubling from the right, then add the ones with a 1 under them:

64	32	16	8	4	2	1
1	0	1	1	0	1	0

$$64 + 16 + 8 + 2 = 90$$

2 marks: 1 for correct place values, 1 for the total. A 7-bit number, so the leftmost place is $2^6 = 64$.

Q19

[3]

Answer. $2AF_{16} = 1010101111_2$

Working. Hexadecimal is base 16 and $16 = 2^4$, so each hex digit is exactly 4 bits. Convert them one at a time – there is no need to go through decimal:

Hex digit	Decimal	4 bits
2	2	0010
A	10	1010
F	15	1111

Join them in the same order:

0010 1010 1111

Leading zeros may be dropped, giving 1010101111_2 (both forms accepted).

Check: $2 \times 256 + 10 \times 16 + 15 = 512 + 160 + 15 = 687$, and $1010101111_2 = 512 + 128 + 32 + 8 + 4 + 2 + 1 = 687$.

✓

3 marks: 1 per digit converted correctly, provided they are joined in order.

Part E — Flowcharts and Algorithms

23 marks

Q20

[5]

Answer. The program prints **2, 3, 5, 6, 8, 9**.

Working. Both lists are already in order. The first loop compares the front item of each list and prints the smaller one, then steps only *that* list forward. $A = (2, 5, 9)$ and $B = (3, 6, 8)$, counting from $i = 0$ and $j = 0$:

i	j	Test	Output	then add 1 to
0	0	$2 < 3$? Yes	print 2	i
1	0	$5 < 3$? No	print 3	j
1	1	$5 < 6$? Yes	print 5	i
2	1	$9 < 6$? No	print 6	j
2	2	$9 < 8$? No	print 8	j
2	3	second loop	print 9	i

After the first loop j has reached 3 (list B is used up), so the second loop prints what is left of A – just the 9. The third loop runs zero times.

What it does: it merges two sorted lists into one sorted list – the output 2, 3, 5, 6, 8, 9 is in order. This is the merge step of merge sort.

5 marks: 3 for the i/j columns tracked correctly, 2 for the output in the right order.

Q21

[4]

Answer. The program prints **31**.

Working. x starts at $n \div 2$ and is then replaced six times by the average of x and $n \div x$. **Every division here is whole-number division** – throw the remainder away each time:

Pass	New $x = (x + 1000 // x) // 2$	x
start	$x = 1000 \div 2$	500
1	$(500 + 2) \div 2 = 502 \div 2$	251
2	$(251 + 3) \div 2 = 254 \div 2$	127
3	$(127 + 7) \div 2 = 134 \div 2$	67
4	$(67 + 14) \div 2 = 81 \div 2$	40
5	$(40 + 25) \div 2 = 65 \div 2$	32
6	$(32 + 31) \div 2 = 63 \div 2$	31

After 6 passes the loop test $i \leq 6$ fails and $x = 31$ is printed.

What it does: it closes in on the square root of n . $\sqrt{1000} = 31.6\dots$, and the whole-number answer is 31 – reached by pass 6 and stable after that.

4 marks: 3 for the six values of x in order, 1 for the printed answer. Losing the remainder at any step derails everything after it, so mark the first wrong row and follow through.

Q22

[2]

Answer. Two boxes are wrong: the loop test, and the position of the print box.

Working. Run the chart on the given numbers and compare:

Numbers read	Should print	Actually prints
30, 25, 15, 20, 10	10	30, 25, 15, 15

Two separate symptoms, so two separate faults.

Bug 1 – the loop stops one number early.

The test reads $i < n$, so with $n = 5$ the loop body runs for $i = 1, 2, 3, 4$ only: the fifth number (10) is never read. That is why the smallest found is 15 and not 10. **Fix:** change the test to $i \leq n$.

Bug 2 – the print box is inside the loop.

“Print smallest” sits within the loop, so it runs on every pass and the program prints four times (30, 25, 15, 15) instead of once. **Fix:** move the print box *after* the loop.

With both fixed the program prints **10**, once.

2 marks: 1 per bug – the mark needs both the box named *and* the correction. Naming only the symptom (“it prints too many times”) scores 0.

Q23

[2]

Answer. One box is wrong: `avg = total // i` should be `avg = total // n`.

Working. Nothing in this chart *looks* wrong, so trace it:

Numbers read	Should print	Actually prints
10, 20, 30, 40	$100 / 4 = 25$	$100 / 5 = 20$
20, 30, 10, 40, 20	$120 / 5 = 24$	$120 / 6 = 20$

Both sets print 20, and both are too low – so the totals are right and the division is not.

Why. The loop ends when the test $i \leq n$ *fails*, which means i has already passed n : after reading 4 numbers, i is **5**, not 4. The divide box uses i , so the total is shared among one person too many:

$$100 \div 5 = 20 \quad \text{instead of} \quad 100 \div 4 = 25$$

Fix: divide by n – the count that was read in – not by the loop counter i .

2 marks: 1 for identifying the divide box, 1 for the correction. This is the off-by-one that a student can only find by tracing, so credit a correct trace that reaches the box even if the wording of the fix is loose.

Q24

[7]

Answer. Flowchart below; it prints **120** for $n = 5$ and **720** for $n = 6$.

Working. **The flowchart** (7 marks: 5 for the chart, 2 for the two predictions). A correct chart needs, in order:

1. **Start** (oval).
2. **Read** n (parallelogram).

3. Set $f = 1$ and $i = 1$ (process). f must start at 1, not 0 – anything times 0 stays 0.
4. A **decision** $i \leq n$ (diamond).
5. On **Yes**: $f = f \times i$, then $i = i + 1$, then an arrow *back* to the decision.
6. On **No**: **print** f (parallelogram), then **Stop** (oval).

Marks are for the loop-back arrow and the counter update as much as the boxes – without $i = i + 1$ the loop never ends.

The predictions. Trace the multiplication:

$$n = 5: 1 \times 1 \times 2 \times 3 \times 4 \times 5 = \mathbf{120}$$

$$n = 6: \mathbf{120} \times 6 = \mathbf{720}$$

The second follows from the first – a student who multiplies out again from scratch is not wrong, just slower.

Q25

[3]

Answer. Flowchart below.

Working. Read the wording one sentence at a time; each becomes a box.

1. **Start** (oval).
2. **Read** price, **read** quantity (parallelograms, or one box reading both).
3. $total = price \times quantity$ (process).
4. A **decision**: $total > 1000$? (diamond).
5. On **Yes**: $total = total - (total \times 10 \div 100)$ – a 10% discount. Equivalent forms such as $total = total \times 0.9$ are fully correct.
6. On **No**: nothing – the arrow joins straight back.
7. **Print** total, then **Stop**.

3 marks: 1 for reading the inputs and computing the total, 1 for a correctly shaped decision with both branches rejoining, 1 for the discount arithmetic and the print.

Note: there is *no loop* in this one. The commonest error is discounting on the No branch as well, which gives every customer 10% off.

- End of Answer Key -